kscript v0.3.1

a programming language

Cade Brown

2025-10-14

Contents

1 kscript: a programming language **2**
1.1 FAQs . . . 3

2 kscript: A Practical Guide **4**
2.1 Chapter 0: Hello World . . . 4
2.2 Chapter 1: Some Basic Uses . . . 4
2.3 Chapter 2: Writing Reusable Functions . . . 4
2.4 Chapter 3: Implementing Custom Types . . . 4

3 Programming Philosophy **4**
3.1 The Octologue: Rules of Thumb . . . 5
3.2 Duck Typing Patterns . . . 7
3.2.1 Magic Attributes . . . 7
3.2.2 Operator Overloading . . . 8

4 The Standard Library **10**
4.1 Global Builtins . . . 11
4.2 m: Mathematical Utilities . . . 17
4.3 nx: NumeriX Tensor Library . . . 20
4.3.1 nx.la: Linear Algebra . . . 22
4.3.2 nx.fft: Fast Fourier Transforms . . . 23
4.4 os: Operating System . . . 25
4.5 io: Input/Output . . . 29
4.6 net: Networking . . . 32
4.6.1 net.http: Hyper Text Transfer Protocol . . . 33
4.7 ffi: Foreign Function Interface . . . 35
4.8 time: Times and Dates . . . 39
4.9 util: Common Utilities . . . 41
4.10 gram: Grammar Regonition . . . 42

5 Complete Syntax Reference **44**
5.1 Formal Grammar EBNF . . . 44
5.2 Expressions . . . 48
5.2.1 Integer Literal . . . 48
5.2.2 Float Literal . . . 49

5.2.3 Complex Literal . . . 50
5.2.4 String Literal . . . 50
5.2.5 List Literal . . . 51
5.2.6 Tuple Literal . . . 52
5.2.7 Set Literal . . . 52
5.2.8 Dict Literal . . . 53
5.2.9 Lambda Function Expression . . . 53
5.2.10 Function Definition . . . 53
5.2.11 Type Definition . . . 54
5.2.12 Operators . . . 55
5.3 Statements . . . 55
5.3.1 Expression Statement . . . 56
5.3.2 Assert Statement . . . 56
5.3.3 Cont Statement . . . 56
5.3.4 Break Statement . . . 56
5.3.5 Ret Statement . . . 57
5.3.6 Throw Statement . . . 57
5.3.7 If Statement . . . 58
5.3.8 While Statement . . . 58
5.3.9 For Statement . . . 58
5.3.10 Try Statement . . . 59
5.3.11 Import Statement . . . 59

6 Implementation Details 60

1 kscript: a programming language

kscript is a dynamic programming language with expressive syntax, cross platform support, and a rich standard library. It's like Python, but better.

Right now, you're reading kscript's documentation and reference manual. It aims to show you how to write kscript by example, as well as explain the design philosophy behind it. Also, it dives into internal implementation details, if you're into that sort of thing.

You can find more information about kscript at the following locations:

- ks.cade.io - online documentation (this page)
- ks.cade.io/repl - online REPL that runs in your browser with WASM
- github.com/cadebrown/kscript - source code repository

A few things I decided to reinvent the wheel for:

- hashtable based dictionary, that preserves insertion order and uses different integer sizes depending on the number of entries
- regular expression (regex) engine, which parses and converts them to finite automata for linear time matching
- string formatting engine, which is type-aware and uses dynamic dispatch for custom types
- numeric tensor library, which works a lot like NumPy
  - implements basic dispatch/elementwise application of functions, common linear algebra operations, Fast Fourier Transforms (FFTs), and more
- kscript language parser
  - bytecode compiler, converting high level parse

– bytecode virtual machine, which is the core of the interpreter using a stack-based execution model, with exceptions and scoping

1.1 FAQs

What is kscript?

kscript is a dynamic programming language with expressive syntax, cross platform support, and a rich standard library. Its primary aim is to allow developers to write platform agnostic programs that can run anywhere, and require little or no platform-specific or os-specific code.

The language is object-oriented, duck-typed, and targets the web platform through WASM and WASI. It is a learning project, so expect rough edges. It's fun to write and demonstrates some interesting concepts, so I hope you enjoy it!

You're currently reading the docs, which is a formal specification of the language, builtins, standard library, and semantics.

Why is kscript?

kscript was designed to be a tool useful in many different circumstances – as a computer calculator, as a task-automation language, GUI development language, numerical programming language, and more. A few languages may come to mind – namely Python, tcl, and so forth.

I found that I had issues with some choices the Python team made. Syntactically, I dislike required/syntactically-significant whitespace, and dislike Python's overuse of `:.` . I feel that many Python modules (for example, the operating system module) do not give the best interface to their functionality, and often require the programmer to use platform-specific code. I'd like to conclude this paragraph with a redeeming note – Python has been very successful and I largely do enjoy the language (even though I have my complaints), and even borrow the great ideas Python has had.

Who is kscript?

It's mainly written by me, Cade Brown. I also had some help from my friend, Gregory Croisdale. It was written as I was in college, mainly during lecture time and a bit on weekends when I was particularly obsessed. A pure passion project!

When is kscript?

kscript was developed mainly during my college days. I started it 2020, and worked on it sporadically until late 2021. After life happened, I dusted it off in 2025 and made a final release with its current feature set.

A spiritual successor to kscript is on the way, but will take more time to be release. Stay tuned!

How is kscript?

kscript is implemented in C99 from scratch, largely as a learning exercise. The tokenizer, parser, compiler, interpreter, and standard library are all essentially implemented in terms of the C standard library.

The implementation is somewhat similar to Python's reference implementation, CPython. Types and objects are C-style structures with additional metadata and function pointers to implement methods. Memory management is done with reference counting, and new types can be defined as C structures that are registered with the runtime.

There is also optional support for some common third party libraries, such as readline (for terminal support), libffi (for calling C functions in shared libraries), and others. In addition, there is platform support for web browers, made possible through Emscripten and WebAssembly (WASM).

2 kscript: A Practical Guide

How might you actually use kscript? Let's explore writing some simple programs to introduce you to the language, its syntax, and its semantics.

This guide will start out simple, explaining new concepts as they are naturally introduced. The goal is to grow in complexity, w

2.1 Chapter 0: Hello World

In honor of tradition, here's the classic "Hello, World!" program, written in kscript:

```
#!repl
print("hello, world")
```

Let's break it down:

- `print` is a builtin function that prints its arguments to standard output (typically, your terminal or console), along with a newline
- `"hello, world"` is a string literal, which is a sequence of characters enclosed in double quotes

And that's it!

2.2 Chapter 1: Some Basic Uses

2.3 Chapter 2: Writing Reusable Functions

2.4 Chapter 3: Implementing Custom Types

```
type Person {
    func __init(self, name, age) {
        self.name = name
        self.age = age
    }

    func greet(self) {
        print("Hello, my name is " + self.name + " and I am " + str(self.age) + " years
    }
}
```

3 Programming Philosophy

If I may be indulged, I'd like to describe the philosophy behind kscript. This section covers the reasoning and goals of kscript, and what design decisions were made to achieve those goals. At a high level, kscript is meant to be written and read as close to pseudocode and mathematical

notation as possible. In other words, very little boiler plate is required, and instead the focus is on the core logic that code is trying to express.

Performance is not the highest priority, but rather correctness, readability, and maintainability. kscript is meant to be a language that is easy to pick up and use for small tasks, but also powerful enough to be used for larger projects. It is also meant to be a language that is easy to learn, with a gentle learning curve. Minimizing extra "noise" is essential as well for REPL-like usage, where you want to quickly write and get output from your code without having to coax the language too much.

3.1 The Octologue: Rules of Thumb

These are the pillars of kscript, which the kscript standard library and all other kscript code should try to adhere to.

Thou Shalt Have No Languages Before KSCRIPT
: When writing a package, or any code in kscript, do not prioritize other languages. Just because you are wrapping a C library does not mean other developers which use your code should feel like they're writing C code.

  Any abstractions should be and should encourage well written and conventional kscript code; anything else adds to the mental strain present in that language to the already-existing mental strain of solving whatever problem they are using your code for, so don't make it harder on them.

Thou Shalt Not Take The Name Of KSCRIPT In Vain
: Don't use kscript to do bad things to people; kscript has no place being used for racist, misogynistic, anti-LGBT+, or any other evil purposes. This should be a bare minimum of anyone doing anything.

Thou Shalt Share
: kscript was founded and written using the principles of free software, and we ask that you do the same for others as well. That way, everyone wins.

  It is well known that free and open source software is more secure, encourages contributions, and is more robust. There is typically little benefit to restricting access to source code nowadays, as it doesn't even make sense financially (it makes more sense to monetize products in other ways). Regardless, we still take a stand that free and open source software is better.

Honour Thy Father And Thy Mother (Technology)
: While kscript may seek to solve problems created by languages and technologies that came before it, we must also realize that a great deal of work has been done in these technologies. It would be foolish to ignore the lessons learned, and block ourselves out from the established programming languages, libraries, and operating systems.

  Therefore, kscript developers, package developers, and users should make code work and be interoperable, when possible, with other technology solutions.

Thou Shalt Not Write Incorrect Code
: Writing code which does not work, or does not work reliably (depending on an implementation is the same as depending on the wind – don't do it!) is a mortal sin in kscript. kscript should give you the tools to make everything cross-platform and generic. If it doesn't, then it's a problem with kscript and you should contact the developers immediately. Otherwise, it's on you!

- Writing OS-specific code is incorrect (in most cases)
- Writing code which requires quirks of a specific architecture is incorrect code (in most cases)

Thou Shalt Not Write Confusing Code

The highest function of code is to solve problems. Although you may complete a task with small, undocumented, and poorly written code, you create more problems than you solve. By assuming things about input to a function (for example), you may simplify your task, but the number of problems does not go down; quite the opposite in fact.

When writing code, you should be good to yourself, as well as those who will end up using your code. This involves giving classes, modules, functions, and even variables accurate names.

This also includes formatting your code (possibly with an IDE or linter). Whereas some languages beat their users down with a stick named 'relevant whitespace', kscript believes that developers are not children, although they may do childish things. By all accounts, you SHOULD indent your code regularly, with consistent use of either spaces or tabs (preferably 4 spaces). But kscript isn't going to stop you from doing otherwise.

Thou Shalt Not Repeat Thyself

Your code should never repeat itself. Your code should adhere to the DRY principle.

This includes at a micro level:

```
# BAD
if A {
    B = 2 * 3
    C = A + B
} else {
    B = 2 * 3
    C = other / B
}

# GOOD. KSCRIPT IS PLEASED
B = 2 * 3
if A {
    C = A + B
} else {
    C = other / B
}
```

As well as at a macro level; The following code should never be written:

```
func my_min(*args) {
    if !args, ret 0
    _m = args[0]
    for i in args[1:] {
        if i < _m, _m = i
    }
    ret i
}
```

This algorithm has already been implemented as the built-in function `min`. Therefore, in kscript, we program according to the acronym DRY (Don't Repeat Yourself), but also

add another: STPO (Solve The Problem Once). The idea is that macro-problems (such as: 'how to compute the minimum element in a collection?', 'what is the best way to have a logging library?', 'what is the best way to store large arrays?', etc.) should not be solved over-and-over (unless, of course, a better solution becomes apparent), but rather, solved once so that solution may be re-used by everyone.

The goal is not to have many implementations to choose from, but rather to have one implementation that is a clear choice (i.e. that you would be a fool to NOT choose it).

Thou Shalt Apply Judgement

All of these rules are just suggestions (albeit strong, and well reasoned ones). There is, inevitably, some use case that comes along that requires the breaking of these sacred pacts.

People that have strong beliefs are often hypocritical. Just refer to commandment 1 (Thou Shalt Not Take The Name Of KSCRIPT In Vain), and don't be as bad as those links.

3.2 Duck Typing Patterns

Patterns in kscript refer to sets of "magic attributes" and/or methods that objects are expected to have in order to fulfill a certain purpose. Objects that have those attributes and/or methods are said to "fit" a given pattern, and can be used like other objects that fit that pattern – this is the basis of all duck-typed programming languages.

Patterns are a similar concept to what are called interfaces or contracts in other languages, but in practice are much more dynamic, as the developer doesn't need to specify the pattern that the object fits. You can think of interfaces/contracts as a more formal specification, whereas patterns are dynamic and only require that a given attribute/method is available when a function expects it to be. If the attribute/method is unavailable, the function processing it will likely throw an error explaining that it does not have the expected attribute and/or method.

Although the type does not matter for a pattern (objects of any type may fit a pattern), many patterns will have an abstract base type which other types are subtypes of. This is primarily done to simplify code and reduce redundancy (i.e. if all subtypes re-implemented the pattern handling code, then there would be code bloat and duplication). However, this is not required and is only done to simplify the implementation in most cases (or, for some sense of type-hierarchy purity).

Some examples of patterns in the standard library are:

- **Number pattern**: documented by the abstract base type `number`
- **IO pattern**: documented by the abstract base type `io.BaseIO`

3.2.1 Magic Attributes

Duck-typing concept

Magic attributes are the name we use to describe how the standard library (or external packages) inspect objects to determine how to use them for a particular purpose.

For example, when `int(x)` is called, and `x` is an unfamiliar type (for example, a custom type), how does kscript know how to convert it to an integer? Well, for converting to integers, there is a well established magic attribute called `__int`, which is searched for on `x` (for example, `x.__int()` is attempted). If that does not work, there is a secondary magic attribute called

`__integral`, which is then searched (`x.__integral()` is attempted). If both of those fail, then a `TypeError` is thrown with a message explaining that `x` could not be converted to an `int`. However, if one of those did succeed, then its return value is expected to be an integer, and kscript can use it as the return value.

The above was just an example, but it shows how specially named attributes allow for different libraries and programs to communicate and translate objects into known quantities for processing. This section contains examples and commonly used magic attributes that you can use in your own code to write easier-to-use libraries and programs.

`__int()`
: Used for direct conversion to `int`. Should return an `int`. Also see `__integral`.

`__integral()`
: Used for integral value conversion. Should return an `int`. Also see `__int`.

`__bool()`
: Used for conversion to `bool`. Should return a `bool`.

`__bytes()`
: Used for direct conversion to `bytes`. Should return a `bytes`.

`__str()`
: Used for direct conversion to `str`. Should return a `str`.

`__repr()`
: Used for string representation (for example, the `repr` function). Should return a `str`.

`__hash()`
: Used for computing a hash of an object. Should return an `int`.

`__len()`
: Used for computing the length of a container, which is typically the number of elements. Should return an `int`.

3.2.2 Operator Overloading

There are a number of operators in kscript that can be used on builtin types and types in the standard library, such as `+` (addition), `-` (subtraction), and `*` (multiplication). However, you can also use them with custom types, which is covered in this section. Defining semantics for operators is called operator overloading.

Operator overloading in kscript is done via magic attributes. Specifically, there are a few different cases:

- For unary operators, such as `+x`, the magic attribute (in this case, `__pos` for `+`) is searched on `type(x)`. So, `type(x).__pos(x)` is attempted. If no such attribute existed, an `Error` is thrown.
- For binary operators, such as `x+y`, the magic attribute (in this case, `__add` for `+`) is searched on `type(x)` and `type(y)`. So, `type(x).__add(x, y)` is attempted. If no such attribute existed, or the result was `undefined`, then `type(y).__add(x, y)` is attempted. If no such attribute existed, or the result was `undefined`, then an `Error` is thrown.

3.2.2.1 Unary Operators

`+(obj) == (obj).__pos()`
: For unary plus which is a no-op for most types

`-(obj) == (obj).__neg()`
: For negation which flips the sign of a number

`~(obj) == (obj).__sqig()`
: For bitwise negation or complex number conjugation

3.2.2.2 Binary Operators

`(L)+(R) == (L).__add(R)`
: For addition or concatenation

`(L)-(R) == (L).__sub(R)`
: For subtraction

`(L)*(R) == (L).__mul(R)`
: For multiplication or repetition

`(L)@(R) == (L).__matmul(R)`
: For matrix multiplication of tensors

`(L)/(R) == (L).__div(R)`
: For division

`(L)//(R) == (L).__floordiv(R)`
: For floored division

`(L)%(R) == (L).__mod(R)`
: For modulo/remainder

`(L)**(R) == (L).__pow(R)`
: For exponentiation

`(L)<<(R) == (L).__lsh(R)`
: For bitwise left shift

`(L)>>(R) == (L).__rsh(R)`
: For bitwise right shift

`(L)|(R) == (L).__binior(R)`
: For bitwise inclusive OR

`(L)^(R) == (L).__binxor(R)`
: For bitwise exclusive OR

`(L)&(R) == (L).__binand(R)`
: For bitwise AND

`(L)==(R) == (L).__eq(R)`
: For equality checking

`(L)!=(R) == (L).__ne(R)`
: For inequality checking

`(L)<(R) == (L).__lt(R)`
: For less-than comparison

`(L)<=(R) == (L).__le(R)`
: For less-than-or-equal comparison

`(L)>(R) == (L).__gt(R)`
: For greater than comparison

`(L)>=(R) == (L).__ge(R)`
: For greater-than-or-equal comparison

Here's a short example showing how to overload the `+` operator:

```
# example class representing a point in 2d space
type Point2d {
    func __init(self, x, y) {
        self.x = x
        self.y = y
    }

    # adds two 'Point2d' objects, which adds their coordinates
    func __add(L, R) {
        if !isinst(L, Point2d) || !isinst(R, Point2d), ret undefined
        ret Point2d(L.x + R.x, L.y + R.y)
    }
}

# Create two objects
A = Point2d(1, 2)
B = Point2d(5, 5)

# Prints '6 7'
C = A + B
print (C.x, C.y)
```

4 The Standard Library

This section documents the builtin modules in kscript, of which there are plenty! The general philosophy in kscript is to make APIs as cross-platform and backend-independent as possible. What that means is that standard types, functions, and modules put forth names and functionality that might not directly map to a particular existing library – even if kscript uses that library internally to perform the functionality.

You can count on these modules being available in any kscript distribution, and having a reliable API. Unreliable/non-standard functions, types, and variables typically begin with an underscore (_), so be weary if using one of those functions, it might not be available everywhere!

A good example of this is the `os` module, which uses the C standard library to perform tasks, but the functions will have different and sometimes better suited names to what they actually do.

You can access modules by using the `import` statement. For example, `import os` will import the `os` module, and allow you to use `os.<funcname>`

4.1 Global Builtins

Types, functions, and values available everywhere

Builtins in kscript are the types, functions, and values that are available in the global namespace and thus available everywhere without the use of `import` statements. They are the most fundamental objects in kscript and are used most frequently, and so they are also made easiest to remember and type. Although kscript is duck-typed, using the builtin types is often recommended, as they have good performance and easy-to-use APIs.

Since kscript is a purely object-oriented language, everything in kscript is an instance of the `object` type, either directly or indirectly. This means that even types are objects, and can be treated generically as such. So can functions. This is in steep contrast to more static programming languages like C, C++, Rust, and so forth, which allow some limited compile time reflection, but little to no runtime inspection of types. kscript, however, is completely dynamic and allows things that other programming languages just can't offer. Here are a few examples:

- Container types (`list`, `tuple`, `dict`, …) can store objects of any types, and multiple objects of different types
- Iterable types may yield objects of any types, and multiple objects of different types
- Objects which are not `func` instances may also be callable, if they implement the `__call` attribute

`object()`
: The most generic type, which is the common type that all other types are derived from. The code `isinst(x, object)` always returns true, no matter what object `x` refers to.

  Objects can be created via the constructor, `object()`, which returns a blank objects that has writeable and readable attributes

`type(obj)`
: This type is what other types are an instance of. `type` is also an instance of `type`.

  Unlike most other types, the code `type(x)` does not created a new type; rather, it returns the type of `x`

`number(obj=0)`
: This type is the abstract base type of all other builtin numeric types (`int`, `bool`, `float`, `complex`).

  By default, `number()` will return the integer `0`. You can pass it any numeric type and it will return one of the builtin numeric types (or throw an error if there was a problem). You can also call it with a string, and it will parse the string and return a builtin numeric type.

  This type also defines and implements stubs for the "number pattern", which dictates how numeric types should behave. Specifically, an object `obj` is said to follow the number pattern if at least one of the following magic attributes holds:

  - if `obj.__integral()` exists, `obj` is assumed to be an integer, and this method should return an `int` with an equivalent numerical value
  - if `obj.__float()` exists, `obj` is assumed to be a real floating point number, and this method should return a `float` with an equivalent numerical value
  - if `obj.__complex()` exists, `obj` is assumed to be a complex floating point number, and this method should return a `complex` with an equivalent numerical value

`int(obj=0, base=none)`

This type describes integers (i.e. whole numbers). This type is a subtype of `number`, and subscribes to the `number` pattern. You can create integers with integer literals, or through using this type as a constructor. If `obj` is a string, then `base` can also be given, which is an integer describing the base format that the string is in.

Some languages (C, C++, Java, and so forth) have sized integer types which are limited to machine precisions (see: here). For example, a 32 bit unsigned integer may only represent the values from 0 to 2^32-1. However, in kscript, all integers are of arbitrary precision – which means they may store whatever values can fit in the main memory of your computer (which is an extremely large limit on modern computers, and you are unlikely to hit that limit in any practical application).

`bool(obj=false)`

This type describes booleans. There are two boolean values, `true` and `false`. Sometimes, these are referred to as `0/1`, `on/off`, or even `yes/no`. `true` and `false` are keywords which result in these values.

You can convert an object to a boolean (its "truthiness", or "truth" value) via this type as a function. For example, `bool(x)` turns `x` into its truthiness value, or, equivalently, `x as bool`. Typically, types that have custom truthiness logic work as expected – numbers convert to `true` if they are non-zero, containers convert to `true` if they are non-empty, and so on. In general, if `bool(x) == true`, then `x` is non-empty, non-zero, and/or valid. Otherwise, it is empty, zero, or perhaps invalid. Specific types may overload it in a way that makes sense for them, so always beware of types that override this functionality.

This conversion to bool is dictated by the `__bool` magic attribute.

Examples:

```
false
>>> bool(false)
false
>>> bool(true)
true
>>> bool(0)
false
>>> bool(1)
true
>>> bool(255)
true
>>> bool('')
false
>>> bool('AnyText')
true
```

`float(obj=0.0, base=none)`

This type describes floating point numbers. This type is a subtype of `number`, and subscribes to the `number` pattern. You can create floats with float literals, or through using this type as a constructor. If `obj` is a string, then `base` can also be given, which is an integer describing the base format that the string is in.

See: Float Literal for creating literals

In addition to real numbers, this type can also represent infinity (positive and negative)

(via `inf` and `-inf`), as well as not-a-number (via `nan`) values.

`float.EPS`
The difference between 1.0 and the next largest number

```
>>> float.EPS
2.22044604925031e-16
```

`float.MIN`
The minimum positive value representable as a `float`

```
>>> float.MIN
2.2250738585072e-308
```

`float.MAX`
The maximum positive (finite) value representable as a `float`

```
>>> float.MAX
1.79769313486232e+308
```

`float.DIG`
The number of significant digits that can be stored in a `float`

```
>>> float.DIG
15
```

`complex(obj=0.0+0.0i)`
This type describes a complex number, with a real and imaginary components (which can take on any `float` values). You can access those elements via the `.re` and `.im` attributes, which will result in `float` objects.

See: Complex Literal for creating literals

`complex.re`
Real component of a complex number, as a `float`

```
>>> (2 + 3i).re
2.0
>>> (3i).re
0.0
>>> (2 + 0i).re
2.0
```

`complex.im`
Imaginary component of a complex number, as a `float`

```
>>> (2 + 3i).im
3.0
>>> (3i).im
3.0
>>> (2 + 0i).im
0.0
```

`str(obj='')`
This type describes a string, which is a sequence of 0-or-more Unicode characters. It is the basic unit of textual information in kscript, and can store any Unicode sequence. All operations are in terms of characters (also called codepoints), and not neccessarily bytes.

Some languages have a different type for a single character and a string. However, in kscript, a character is simply a string of length 1. And, the empty string is the string of length 0. Additionally, strings are immutable (which means they cannot be changed). For example, `x[0] = 'c'` will throw an error in kscript. Instead, you should use slicing and re-assign to the same name: `x = 'c' + x[1:]`.

You can create a string via calling this type as a function with an argument (default: empty string). For non-`str` objects, conversion depends on the `__str` magic attribute, which is expected to return a `str`.

See String Literal for creating literals.

Internally, kscript uses UTF-8 to store the textual information.

Strings can be added together with the `+` operator, which concatenates their contents. For example, `'abc' + 'def' == 'abcdef'`

`str.upper(self)`
Computes an all-upper-case version of `self`

`str.lower(self)`
Computes an all-lower-case version of `self`

`str.isspace(self)`
Computes whether all characters in `self` are space characters

`str.isprint(self)`
Computes whether all characters in `self` are printable characters

`str.isalpha(self)`
Computes whether all characters in `self` are alphabetical characters

`str.isnum(self)`
Computes whether all characters in `self` are numeric characters

`str.isalnum(self)`
Computes whether all characters in `self` are alphanumeric

`str.isident(self)`
Computes whether `self` is a valid kscript identifier

`str.startswith(self, obj)`
Computes whether `self` starts with `obj` (which may be a string, or a tuple of strings)

`str.endswith(self, obj)`
Computes whether `self` ends with `obj` (which may be a string, or a tuple of strings)

`str.join(self, objs)`
Computes a string with `self` in between every element of `objs` (converted to strings)

`str.split(self, by)`
Returns a `list` of strings created when splitting `self` by `by` (which may be a string of tuple of seperators)

`str.index(self, sub, start=none, end=none)`
Find a substring within `self[self:end]`, or throw an error if it was not found

`str.find(self, sub, start=none, end=none)`
Find a substring within `self[self:end]`, or return `-1` if it was not found

str.replace(self, sub, by)
Returns a string with instances of `sub` replaced with `by`

str.trim(self)
Trims the beginning and end of `self` of spaces, and returns what is left

input(prompt='')
This function reads a line of text from standard input (typically, your terminal or console). If `prompt` is given, it is printed to standard output before reading input.

KeyError(what='')
Errors of this type are thrown when any of the following things occur:

- A key to a container (for example, a `dict`) is not found when searched
- A key to a container is invalid (for example, `dict` keys must be `hash`-able)

Examples:

```
>>> x = { 'a': 1, 'b': 3 }
{ 'a': 1, 'b': 3 }
>>> x['c']
KeyError: 'c'
Call Stack:
  #0: In '<inter-2>' (line 1, col 1):
x['c']
^~~~~
In <thread 'main'>
```

IndexError(what='')
Errors of this type are thrown when any of the following things occur:

- A index to a sequence (for example, a `list`) is out of range

Examples:

```
>>> x = ['a', 'b']
['a', 'b']
>>> x[3]
KeyError: Index out of range
Call Stack:
  #0: In '<inter-5>' (line 1, col 1):
x[2]
^~~
In <thread 'main'>
```

This type is a subtype of the `KeyError` type

ValError(what='')
Errors of this type are thrown when a value provided does not match what is expected

Examples:

```
>>> nan as int
ValError: Cannot convert 'nan' to int
Call Stack:
  #0: In '<expr>' (line 1, col 1):
nan as int
```

```
^~~~~~~~~~
  #1: In int.__new(self, obj=none, base=10) [cfunc]
In <thread 'main'>
```

AssertError(what='')

Errors of this type are thrown when an `assert` statement has a falsey conditional

Examples:

```
>>> assert false
AssertError: Assertion failed: 'false'
Call Stack:
  #0: In '<inter-0>' (line 1, col 1):
assert false
^~~~~~~~~~~~
In <thread 'main'>
```

MathError(what='')

Errors of this type are thrown when a mathematical operation is given invalid or out of range operands

Examples:

```
>>> 1 / 0
MathError: Division by 0
Call Stack:
  #0: In '<expr>' (line 1, col 1):
1 / 0
^~~~
  #1: In number.__div(L, R) [cfunc]
In <thread 'main'>
>>> import m
>>> m.sqrt(-1)
MathError: Invalid argument 'x', requirement 'x >= 0' failed
Call Stack:
  #0: In '<expr>' (line 1, col 1):
m.sqrt(-1)
^~~~~~~~~~
  #1: In m.sqrt(x) [cfunc]
In <thread 'main'>
>>> m.sqrt(-1 + 0i)  # Make sure to pass a 'complex' in if you want complex output
1.0i
```

ArgError(what='')

Errors of this type are thrown when arguments to a function do not match the expected number, or type

Examples:

```
>>> ord('a', 'b')
ArgError: Given extra arguments, only expected 1, but given 2
Call Stack:
  #0: In '<expr>' (line 1, col 1):
ord('a', 'b')
```

```
^~~~~~~~~~~~
  #1: In ord(chr) [cfunc]
```

SizeError(what='')
Errors of this type are thrown when arguments are of invalid sizes/shapes

OSError(what='')
Errors of this type are thrown when an error is reported by the OS, for example by setting `errno` in C

This is a templated type, which means there are subtypes based on the type of error expressed. The specific templated types are sometimes platform-specific, and we are currently working to standardize what we can.

Examples:

```
>>> open("NonExistantFile.txt")
OSError[2]: Failed to open 'NonExistantFile.txt' (No such file or directory)
Call Stack:
  #0: In '<expr>' (line 1, col 1):
open("NonExistantFile.txt")
^~~~~~~~~~~~~~~~~~~~~~~~~~
  #1: In open(src, mode='r') [cfunc]
  #2: In io.FileIO.__init(self, src, mode='r') [cfunc]
In <thread 'main'>
```

4.2 m: Mathematical Utilities

Math Module

This module, `m`, provides functionality to aid in mathematical problems/needs. This module contains common mathematical constants (such as , , e, and so forth), as well as functions that efficiently and accurately compute commonly used functions (such as sin, cos, Γ, and so forth). This module also includes some integer and number-theoretic functions, such as computing the greatest common denominator (GCD), binomial coefficients, and primality testing.

This module is meant to work with the types that follow the `number` pattern, such as `int`, `float`, and `complex`. Most functions are defined for real and complex evaluation. If a real number is given, then (generally) a real number is returned. If a complex number is given, then (generally) a complex number is returned. If a real number is given (for example, to `m.sqrt`) and the result would be a complex number (i.e. `m.sqrt(-1)`), then an error is thrown (this makes it easy to find bugs, and in general real numbers are what most people care about – and they would like an error on code such as `m.sqrt(-1)`). To get around this, you can write: `m.sqrt(complex(-1))`, and the result will always be a complex, and an error won't be thrown for negative numbers.

Constants are given in maximum precision possible within a float, but for all of these constants, their value is not exact. This causes some issues or unexpected results. For example, mathematically,

$$\sin(\pi) = 0$$

, but `m.sin(m.pi) == 1.22464679914735e-16`. This is expected when using finite precision. Just make sure to keep this in mind.

Here are some recommended ways to handle it:

```
>>> if m.sin(m.pi) == 0 {}                # Bad, may cause unexpected results
>>> if abs(m.sin(m.pi) - 0) < 1e-6 {}     # Better, uses a decent tolerance (1e-6 is pret
>>> if m.isclose(m.sin(m.pi), 0) {}       # Best, use the m.isclose() function
```

m.pi
The value of , as a `float`

```
>>> m.pi
3.141592653589793
```

m.tau
The value of , as a `float`

```
>>> m.tau
6.283185307179586
```

m.e The value of e, as a `float`

```
>>> m.e
2.718281828459045
```

m.mascheroni
The value of the Euler–Mascheroni constant, as a `float`

```
>>> m.mascheroni
0.577215664901533
```

m.isclose(x, y, abs_err=1e-6, rel_err=1e-6)
Computes whether `x` and `y` are "close", i.e. within `abs_err` absolute error or having a relative error of `rel_err`.

Is equivalent to:

```
func isclose(x, y, abs_err=1e-6, rel_err=1e-6) {
    ad = abs(x - y)
    ret ad <= abs_err || ad <= abs_err * max(abs(x), abs(y))
}
```

m.floor(x)
Computes the floor of `x`, as an `int`

m.ceil(x)
Computes the ceiling of `x`, as an `int`

m.round(x)
Computes the nearest `int` to `x`, rounding towards `+inf` if exactly between integers

m.sgn(x)
Computes the sign of `x`, returning one of `+1`, `0`, or `-1`

m.sqrt(x)
Computes the square root of `x`.

If `x` is a real type (i.e. `int` or `float`) then negative numbers will throw a `MathError`. You can instead write: `m.sqrt(complex(x))`, which will give complex results for negative numbers

m.exp(x)
Computes the expontial function (base-e) of `x`

`m.log(x, b=m.e)`
Computes the logarithm (base-b) of `x`. Default is the natural logarithm

`m.rad(x)`
Converts `x` (which is in degrees) to radians

`m.deg(x)`
Converts `x` (which is in radians) to degrees

`m.hypot(x, y)`
Computes the side of a right triangle with sides `x` and `y`

`m.sin(x)`
Computes the sine of `x` (which is in radians)

`m.cos(x)`
Computes the cosine of `x` (which is in radians)

`m.tan(x)`
Computes the tangent of `x` (which is in radians)

`m.sinh(x)`
Computes the hyperbolic sine of `x` (which is in radians)

`m.cosh(x)`
Computes the hyperbolic cosine of `x` (which is in radians)

`m.tanh(x)`
Computes the hyperbolic tangent of `x` (which is in radians)

`m.asin(x)`
Computes the inverse sine of `x` (which is in radians)

`m.acos(x)`
Computes the inverse cosine of `x` (which is in radians)

`m.atan(x)`
Computes the inverse tangent of `x` (which is in radians)

`m.asinh(x)`
Computes the inverse hyperbolic sine of `x` (which is in radians)

`m.acosh(x)`
Computes the inverse hyperbolic cosine of `x` (which is in radians)

`m.atanh(x)`
Computes the inverse hyperbolic tangent of `x` (which is in radians)

`m.erf(x)`
Computes the error function of `x`

Defined as

$$\frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2} dt$$

`m.erfc(x)`
Computes the complimentary error function of `x`, defined as `1 - m.erf(x)`

m.gamma(x)
Computes the Gamma function of x,

$$\Gamma(x)$$

m.zeta(x)
Computes the Riemann Zeta function of x,

$$\zeta(x)$$

m.modinv(x, n)
Computes the modular inverse of x within the ring of integers modulo n (i.e.

$$Z_n$$

)

A MathError is thrown if no such inverse exists

m.gcd(x, y)
Computes the Greatest Common Divisor (GCD) of x and y

m.egcd(x, y)
Computes the Extended Greatest Common Divisor (EGCD) of x and y, returning a tuple of (g, s, t) such that x*s + y*t == g == m.gcd(x, y)

If abs(x) == abs(y), then (g, 0, m.sgn(y)) is returned

4.3 nx: NumeriX Tensor Library

The NumeriX module (nx) provides array operations, linear algebra algorithms, transforms, and other numerical algorithms. It is similar to the math module (m), but is intended to work for specific datatypes and compute the result in parallel for entire arrays. This is sometimes called array programming.

You can count on these modules being available in any kscript distribution, and having a reliable API. Unreliable/non-standard functions, types, and variables typically begin with an underscore (_), so be weary if using one of those functions, it might not be available everywhere!

A good example of this is the os module, which uses the C standard library to perform tasks, but the functions will have different and sometimes better suited names to what they actually do.

You can access modules by using the import statement. For example, import os will import the os module, and allow you to use os.<funcname>

nx.abs(x, r=none)
Computes elementwise absolute value of x, storing in r (default: return new array)

nx.conj(x, r=none)
Computes elementwise conjugation of x, storing in r (default: return new array)

nx.neg(x, r=none)
Computes elementwise negation of x, storing in r (default: return new array)

nx.add(x, y, r=none)
Computes elementwise addition of x and y, storing in r (default: return new array)

nx.sub(x, y, r=none)
Computes elementwise subtraction of `x` and `y`, storing in `r` (default: return new array)

nx.mul(x, y, r=none)
Computes elementwise multiplication of `x` and `y`, storing in `r` (default: return new array)

nx.div(x, y, r=none)
Computes elementwise division of `x` and `y`, storing in `r` (default: return new array)

nx.floordiv(x, y, r=none)
Computes elementwise floored division of `x` and `y`, storing in `r` (default: return new array)

nx.mod(x, y, r=none)
Computes elementwise modulo of `x` and `y`, storing in `r` (default: return new array)

nx.pow(x, y, r=none)
Computes elementwise power of `x` and `y`, storing in `r` (default: return new array)

nx.exp(x, r=none)
Computes elementwise exponential function of `x`, storing in `r` (default: return new array)

nx.log(x, r=none)
Computes elementwise natural logarithm of `x`, storing in `r` (default: return new array)

nx.sqrt(x, r=none)
Computes elementwise square root of `x`, storing in `r` (default: return new array)

nx.min(x, axes=none, r=none)
Computes reduction on the minimum of `x` on `axes` (default: all), storing in `r` (default: return new array)

nx.max(x, axes=none, r=none)
Computes reduction on the maximum of `x` on `axes` (default: all), storing in `r` (default: return new array)

nx.sum(x, axes=none, r=none)
Computes reduction on the sum of `x` on `axes` (default: all), storing in `r` (default: return new array)

nx.prod(x, axes=none, r=none)
Computes reduction on the product of `x` on `axes` (default: all), storing in `r` (default: return new array)

nx.cumsum(x, axis=-1, r=none)
Computes cumuluative sum of `x` on `axes` (default: last), storing in `r` (default: return new array)

nx.cumprod(x, axis=-1, r=none)
Computes cumulative product of `x` on `axes` (default: last), storing in `r` (default: return new array)

nx.sort(x, axis=-1, r=none)
Sorts `x` on `axis` (default: last), storing in `r` (default: return new array)

nx.cast(x, dtype, r=none)
Casts `x` to a datatype, storing in `r` (default: return new array)

nx.fpcast(x, dtype, r=none)
Casts `x` to a datatype, storing in `r` (default: return new array). This is like `nx.cast`,

except that it automatically converts to and from fixed point and floating point. For example, if going from integer to float types, the result is scaled to the `[0, 1]` range (unsigned values) or `[-1, 1]` range (signed values). Likewise, going from float to integer types, the result is scaled to the full range of the integer type, from the floating point scale.

4.3.1 nx.la: Linear Algebra

This module is a submodule of the `nx` module. Specifically, it implements functionality related to dense linear algebra.

Most functions operate on arrays, which are expected to be of rank-2 or more (in which case it is a stack of matrices).

`nx.la.norm(x)`

Computes the Frobenius norm of `x` (which should be of rank-2 or more)

Equivalent to `nx.sqrt(nx.sum(nx.abs(x) ** 2, (-2, -1)))`

`nx.la.diag(x)`

Creates a diagonal matrix with `x` as the diagonal. If `x`'s rank is greater than 1, then it is assumed to be a stack of diagonals, and this function returns a stack of matrices

Examples:

```
>>> nx.la.diag([1, 2, 3])
[[1.0, 0.0, 0.0],
 [0.0, 2.0, 0.0],
 [0.0, 0.0, 3.0]]
>>> nx.la.diag([[1, 2, 3], [4, 5, 6]])
[[[1.0, 0.0, 0.0],
  [0.0, 2.0, 0.0],
  [0.0, 0.0, 3.0]],
 [[4.0, 0.0, 0.0],
  [0.0, 5.0, 0.0],
  [0.0, 0.0, 6.0]]]
```

`nx.la.perm(x)`

Creates a permutation matrix with `x` as the row interchanges. If `x.rank > 1`, then it is assumed to be a stack of row interchanges, and this function returns a stack of matrices

Equivalent to `nx.onehot(x, x.shape[-1])`

Examples:

```
>>> nx.la.perm([0, 1, 2])
[[1.0, 0.0, 0.0],
 [0.0, 1.0, 0.0],
 [0.0, 0.0, 1.0]]
>>> nx.la.perm([1, 2, 0])
[[0.0, 1.0, 0.0],
 [0.0, 0.0, 1.0],
 [1.0, 0.0, 0.0]]
```

`nx.la.matmul(x, y, r=none)`

Calculates the matrix product `x @ y`. If `r` is none, then a result is allocated, otherwise

it must be the correct shape, and the result will be stored in that.

Expects matrices to be of shape:

- `x`: `(..., M, N)`
- `y`: `(..., N, K)`
- `r`: `(..., M, K)` (or `r==none`, it will be allocated)

Examples:

```
>>> nx.la.matmul([[1, 2], [3, 4]], [[5, 6], [7, 8]])
[[19.0, 22.0],
 [43.0, 50.0]]
```

`nx.la.factlu(x, p=none, l=none, u=none)`

Factors `x` according to LU decomposition with partial pivoting, and returns a tuple of `(p, l, u)` such that `x == nx.la.perm(p) @ l @ u` (within numerical accuracy), and `p` gives the row-interchanges required, `l` is lower triangular, and `u` is upper triangular. If `p`, `l`, or `u` is given, it is used as the destination, otherwise a new result is allocated.

Expects matrices to be of shape:

- `x`: `(..., N, N)`
- `p`: `(..., N)` (or if `p==none`, it will be allocated)
- `l`: `(..., N, N)` (or if `l==none`, it will be allocated)
- `u`: `(..., N, N)` (or if `u==none`, it will be allocated)

Examples:

```
>>> (p, l, u) = nx.la.factlu([[1, 2], [3, 4]])
([1, 0], [[1.0, 0.0],
 [0.333333333333333, 1.0]], [[3.0, 4.0],
 [0.0, 0.666666666666667]])
>>> nx.la.perm(p) @ l @ u
[[1.0, 2.0],
 [3.0, 4.0]]
```

4.3.2 nx.fft: Fast Fourier Transforms

This module is a submodule of the `nx` module. Specifically, it provides functionality related to FFTs (Fast Fourier Transforms).

Different languages/libraries use different conventions for FFT/IFFT/other transforms. It is important the developer knows which are used by kscript, as they are explained per function.

`nx.fft.fft(x, axes=none, r=none)`

Calculates the forward FFT of `x`, upon `axes` (default: all axes), and stores in `r` (or, if `r==none`, then a result is allocated)

The result of this function is always complex. If an integer datatype input is given, the result will be `nx.complexdouble` . All numeric datatypes are supported, and are to full precision.

For a 1-D FFT, let

$$N$$

be the size,

$$x$$

be the input, and

$$X$$

be the output. Then, the corresponding entries of

$$X$$

are given by:

$$X_j = \sum_{k=0}^{N-1} x_k e^{-2\pi ijk/N}$$

For an N-D FFT, the result is equivalent to doing a 1D FFT over each of `axes`

See `nx.fft.ifft` (the inverse of this function)

Examples:

```
>>> nx.fft.fft([1, 2, 3, 4])
[10.0+0.0i, -2.0-2.0i, -2.0+0.0i, -2.0+2.0i]
```

`nx.fft.ifft(x, axes=none, r=none)`

Calculates the inverse FFT of `x`, upon `axes` (default: all axes), and stores in `r` (or, if `r==none`, then a result is allocated)

The result of this function is always complex. If an integer datatype input is given, the result will be `nx.complexdouble` . All numeric datatypes are supported, and are to full precision.

For a 1-D IFFT, let

$$N$$

be the size,

$$x$$

be the input, and

$$X$$

be the output. Then, the corresponding entries of

$$X$$

are given by:

$$X_j = \sum_{k=0}^{N-1} \frac{x_k e^{2\pi ijk/N}}{N}$$

Note that there is a

$$\frac{1}{N}$$

term in the summation. Some other libraries do different kinds of scaling. The kscript fft/ifft functions will always produce the same result (up to machine precision) when given: `nx.fft.ifft(nx.fft.fft(x))`

For an N-D IFFT, the result is equivalent to doing a 1D IFFT over each of `axes`

See `nx.fft.fft` (the inverse of this function)

Examples:

```
>>> nx.fft.ifft([1, 2, 3, 4])
[2.5+0.0i, -0.5+0.5i, -0.5+0.0i, -0.5-0.5i]
```

4.4 os: Operating System

Operating System Module

This module, `os`, allows code to interact with the operating system (OS) that the program is running on. For example, creating directories, gathering information about the filesystem, launching threads, launching processes, and accessing environment variables are all covered in this module.

Due to differences between operating systems that kscript can run on, some functionality may be different, or even missing on some platforms. We attempt to document known cases of functions behaving differently or when something is not supported. Typically, an error is thrown whenever something is not available on a particular platform.

`os.argv`
: The list of commandline arguments passed to the program.

  Typically, `os.argv[0]` is the file that was ran, and `os.argv[1:]` are the arguments given afterward

`os.stdin`
: A readable `io.FileIO` object representing the input to the program

`os.stdout`
: A writeable `io.FileIO` object representing the output from the program

  `os.stderr`

  A writeable `io.FileIO` object representing the error output from the program

`os.getenv(key, defa=none)`
: Retrieves an environment variable corresponding to `key` (which is expected to be a string)

  In the case that `key` did not exist in the environment, this function's behavior depends on whether `defa` is given. - If `defa` is given, then it is returned

  Otherwise, an `OSError` is thrown

`os.setenv(key, val)`
: Sets an environment variable corresponding to `key` (which is expected to be a string) to `val` (which is also expected to be a string)

  In the case that something went wrong (i.e. an invalid name, general OS error), an `OSError` is thrown

`os.cwd()`
: Get the current working directory of the process, as an `os.path` object

  To retrieve it as a string, the code `os.getcwd() as str` can be used

  To change the working directory, see the `os.chdir` function

`os.chdir(path)`
: Set the current working directory of the process to `path`. `path` is expected to be either a string or an `os.path` object

If `path` did not exist, or for some other reason the operation failed (i.e. permissions), an `OSError` is thrown

`os.mkdir(path, mode=0o777, parents=false)`
Creates a directory on the filesystem at `path`, with mode `mode`. Mode is expected to be a octal numerical notation of the file permission bits (default: allow everything for everybody)

If `parents` is truthy, then parents of `path` that do not exist are created recursively. If `parents` is false, then an error is raised when trying to create a directory within another directory that does not exist yet

os.rm(path, parents=false)‘:
Removes a file or directory `path` from the filesystem

If `path` refers to a directory, then the behavior depends on `parents`: - If `parents` is truthy, then non-empty directories have their contents recursively deleted

- Otherwise, `path` will only be removed if it is empty; otherwise this function will throw an `OSError`

`os.listdir(path)`
Returns a tuple of `(dirs, files)` representing the directories and files within the directory `path`, respectively. Note that the elements within the lists `dirs` and `files` are string objects, and not `os.path` objects. The entries `'.'` and `'..'` are never included in `dirs`

If `path` is not a directory, this function throws an `OSError`

`os.glob(path)`
Returns a list of string paths matching `path`, which is a glob. Essentially, wildcard expands `path` to match anything fitting that pattern.

`os.stat(path)`
Queries information about the file or directory `path` on the filesystem. This is a type that can be used as a function to perform such a query

It has the following attributes:

- `.dev`: Device ID, which is typically encoded as major/minor versions. This is OS-specific typically
- `.inode`: The inode of the file or directory on disk as an integer
- `.gid`: The group ID of the owner
- `.uid`: The user ID of the owner
- `.size`: The size, in bytes, of the file
- `.mode`: The mode of the file, represented as a bitmask integer
- `.mtime`: The time of last modification, in seconds-since-epoch, as a floating point number (see the `time` module)
- `.atime`: The time of last access, in seconds-since-epoch, as a floating point number (see the `time` module)
- `.ctime`: The time of last status change, in seconds-since-epoch, as a floating point number (see the `time` module)

`os.lstat(path)`
Queries information about the file or link or directory `path` on the filesystem. Returns an `os.stat` object.

Equivalent to `os.stat`, except for the fact that this function does not follow symbolic links (and thus, if called with a symbolic link, queries information about the link itself, rather than the path that it points to).

`os.fstat(fd)`

Queries information about an open file descriptor `fd` (which is expected to be an integer, or convertible to one). Returns an `os.stat` object

Examples:

```
>>> os.fstat(os.stdin)
<os.stat ...>
>>> os.fstat(os.stdin.fileno) # os.stdin.fileno == os.stdin as int
<os.stat ...>
```

`os.pipe()`

Creates a new pipe, and returns a tuple of `(readio, writeio)`, which are the readable and writeable ends of the pipe, respectively. Both objects are of type `io.RawIO`

`os.dup(fd, to=-1)`

Duplicates an open file descriptor `fd` (which should be an integer, or convertible to one)

If `to < 0`, then this function creates a new file descriptor and returns the corresponding `io.RawIO` object. Otherwise, it replaces `to` with a copy of the source of `fd`

`os.exec(cmd)`

Executes `cmd` (which should either be a string, or a list of strings) as if it was typed into the system shell, and return the exit code as an integer

See `os.proc` type for more fine grain control of process spawning

`os.fork()`

Forks the current process, resulting in two processes running the code afterwards. This function returns `0` in the child process, and the PID (process ID) in the parent process (which should be an integer greater than 0)

This function is available on the following platforms: linux, macos, unix

`os.mutex()`

This function creates a lock which can be used to restrict access to certain data or code. This is a type which can be used as a function to create an instance

Each mutex starts out unlocked, and then can be locked (or attempted to lock via various functions)

`os.mutex.lock(self)`

Locks the mutex, waiting until any other threads which hold the lock to unlock it before it is acquired

Note: If the same thread which is attempting to lock the mutex has already locked it, a "deadlock" may occur and your program may halt

`os.mutex.trylock(self)`

Locks the mutex, if it can be locked instantly, otherwise do nothing. Returns whether it succesfully locked. The mutex should only be unlocked if this function returned true

Example:

```
>>> mut = os.mutex()
>>> if mut.trylock() {
...     # Acquired lock, do stuff...
...     mut.unlock() # Must unlock!
... } else {
...     # Failed to acquire lock, do other stuff...
...     # Do not unlock!
... }
```

os.mutex.unlock(self)
: Unlocks the mutex, waiting until any other threads which hold the lock to unlock it before it is acquired

os.thread(of, args=(), name=none)
: Creates a new thread which runs the the function `of` with arguments `args` (default: no arguments), with the name `name` (default: auto-generate a name). This is a type which can be called like a function to create an instance

  The thread starts out inactive, you must call `.start()` to actually begin executing

  os.thread.start(self)
  : Starts executing the thread

  os.thread.join(self)
  : Waits for a thread to finish executing, blocking until it does

  os.thread.isalive(self)
  : Polls the thread, and returns a boolean indicating whether the thread is currently alive and executing

os.proc(argv)
: Creates a new process with the given arguments `argv`, which can be either a string, or a list of strings representing the arguments. This is a type which can be called to create an instance.

  os.proc.pid
  : The process ID, as an integer

  os.proc.stdin
  : The standard input of the process, as either an `io.RawIO` or `io.FileIO` (depending on launch configuration). This is writeable, and can be used to send input to the process

  os.proc.stdout
  : The standard output of the process, as either an `io.RawIO` or `io.FileIO` (depending on launch configuration). This is readable, and can be used to read output from the process

  os.proc.stderr
  : The standard error output of the process, as either an `io.RawIO` or `io.FileIO` (depending on launch configuration). This is readable, and can be used to read error output from the process

  os.proc.join(self)
  : Waits for the process to finish executing, and returns the exit code of the process as an integer

os.proc.isalive(self)
: Polls the process and returns a boolean describing whether the process is still alive and running

os.proc.signal(self, code)
: Sends an integer signal to the process

os.proc.kill(self)
: Attempts to kill the process by sending the 'KILL' signal (typically `9`) to the process

os.path(path='.', root=none)
: Creates a path object, which acts similarly to a string (i.e. can be passed to functions such as `os.chdir`, `os.stat`, and so on) but also can be manipulated at higher levels than a string, which makes it easier to traverse the filesystem, and makes for more readable code than interpreting `os.stat` results

  A distinction should be made between absolute paths (i.e. those which unambiguously refer to a single file on disk) and relative paths (i.e. those which require a working directory to resolve completely). To convert a string or `os.path` object to its absolute path, you can use the `os.real` function. Or, the builtin `abs` function works on `os.path` objects.

  os.path.root
  : Either `none` (for relative paths), or a string representing the start of an absolute path

    Since some platforms (most Unix-like ones) use `/` as the root for the filesystem, and other platforms (such as Windows) use drive letters to denote absolute paths (`C:\`, `D:\`, etc), the `.root` may be any of those valid roots, depending on which platform you are on. On all platforms, relative paths have `root==none`

    Examples:

    ```
    >>> os.path('/path/to/file.txt').root
    '/'
    >>> os.path('relative/path/to/file.txt').root
    none
    ```

  os.path.parts
  : A tuple containing string parts of the path, which is implicitly seperated by directory seperators.

    Examples:

    ```
    >>> os.path('/path/to/file.txt').parts
    ('path', 'to', 'file.txt')
    >>> os.path('relative/path/to/file.txt').parts
    ('relative', 'path', 'to', 'file.txt')
    ```

4.5 io: Input/Output

The input/output module (`io`) provides functionality related to text and binary streams, and allows generic processing on streams from many sources.

Specifically, it provides file access (through `io.FileIO`), as well as in-memory stream-like objects for text (`io.StringIO`), as well as bytes (`io.BytesIO`). These types have similar interfaces such that they can be passed to functions and operated on generically. For example, you could

write a text processor that iterates over lines in a file and performs a search (like `grep`), and then a caller could actually give a `io.StringIO` and the search method would work exactly the same. Similarly, it is often useful to build up a file by using `io.BaseIO.write()`, so that when outputting to a file, large temporary strings are not being built. However, sometimes you want to be able to build without creating a temporary file – you can substitute an `io.StringIO` and then convert that to a string afterwards.

`io.Seek`

This type is an `enum` of `whence` values for various seek calls (see `io.BaseIO.seek`). Their values are as follows:

`io.Seek.SET`

Represents a reference point from the start of the stream

`io.Seek.CUR`

Represents a reference point from the current position in the stream

`io.Seek.END`

Represents a reference point from the end of the stream

`io.fdopen(fd, mode='r', src=none)`

Opens an integral file descriptor (`fd`) as a buffered IO (a `io.FileIO` object), with a readable name (`src`) (optional)

`io.BaseIO()`

This type is an abstract type that defines the io pattern. The methods listed here can be used on all io-like objects (for example, `io.FileIO`, `io.StringIO`, and so on) and they should all behave according to this pattern and functionality. Therefore, the methods and attributes are only documented here.

You can iterate over io-pattern objects, which iterates through the lines (seperated by `'\n'` characters). For example, to iterate through lines of standard input (`$os.stdin`), you can use:

```
for line in os.stdin {
    # do stuff with 'line'
}
```

`io.BaseIO.read(self, sz=none)`

Reads a message, of up to `sz` length (returns truncated result if `sz` was too large). If size is ommitted, the rest of the stream is read and returned as a single chunk

For text-based IOs, `sz` gives the number of characters to read. For binary-based IOs, `sz` gives the number of bytes to read.

`io.BaseIO.write(self, msg)`

Writes a message to an io

If `msg` does not match the expected type (`str` for text-based IOs, and `bytes` for binary-based IOs), and no conversion is found, then an `IOError` is thrown

`io.BaseIO.seek(self, pos, whence=io.Seek.SET)`

Seek to a given position (`pos`) from a given reference point (see `io.Seek`)

`io.BaseIO.tell(self)`

Returns an integer describing the current position within the stream, from the start (i.e. `0` is the start of the stream)

io.BaseIO.trunc(self, sz=0)
Attempts to truncate a stream to a given size (default: truncate complete to empty)

io.BaseIO.eof(self)
Returns a boolean indicating whether the end-of-file (EOF) has been reached

io.BaseIO.close(self)
Closes the IO and disables further reading/writing

io.BaseIO.printf(self, fmt, *args)
Print formatted text to `self`. Does not include a line break or spaces between arguments

See `printf` for documentation on `fmt` and arguments

io.RawIO(src, mode='r')
Represents an unbuffered io, which is from either a file on disk, or a simulated file (for example, such as the result from `os.pipe()`). The constructor creates one from a file on disk, and behaves similar to the `open` function

This is a subtype of `io.BaseIO`, and implements the pattern fully. Additionally, this type has the following attributes:

io.RawIO.fileno
This attribute retrieves the file descriptor associated with the stream

io.FileIO(src, mode='r')
Represents buffered io, which is from either a file on disk, or a simulated file (for example, such as the result from `os.pipe()`). The constructor creates one from a file on disk, and is equivalent to the `open` function.

This is a subtype of `io.BaseIO`, and implements the pattern fully. Additionally, this type has the following attributes:

io.FileIO.fileno
This attribute retrieves the file descriptor associated with the stream

This is useful for passing to functions that expect a file descriptor (for example, `os.fstat`)

For example, on most systems:

```
>>> os.stdin.fileno
0
>>> os.stdout.fileno
1
>>> os.stderr.fileno
2
```

io.StringIO(obj='')
Represents an IO for textual information, being built in memory (i.e. not as a file on disk). It can be used in places where `io.FileIO` is typically used.

io.BytesIO(obj='')
Represents an IO for byte-based information, being built in memory (i.e. not as a file on disk). It can be used in places where `io.FileIO` is typically used.

4.6 net: Networking

The network module (`net`) provides functionality related to the world wide web, and other networks (i.e. LANs).

`net.FK`
: This type represents the type of family of address/connection/network

  `net.FK.INET4`
  : IPv4 style addresses.

    Addresses for this type of socket are expected to be a `tuple` containing `(host, port)`, where `host` is a string hostname/IP, and `port` is an integer.

  `net.FK.INET6`
  : IPv6 style addresses.

    Addresses for this type of socket are expected to be a `tuple` containing `(host, port, flow, scope)`, where `host` is a string hostname/IP, and `port` is an integer

  `net.FK.BT`: Bluetooth style addresses.

  TODO: This is not yet implemented

`net.SK`
: This type represents the type of socket

  `net.SK.RAW`
  : Raw socket kind, which sends raw packets

  `net.SK.TCP`
  : TCP/IP socket kind, which goes through the TCP/IP network protocol

  `net.SK.UDP`
  : UDP socket kind, which goes through the UDP network protocol

  `net.SK.PACKET`
  : Packet socket kind

    This kind of socket is deprecated

  `net.SK.PACKET_SEQ`
  : Packet (sequential) socket kind

`net.PK`
: This type represents the type of protocol used in transmission

  `net.PK.AUTO`
  : Automatic protocol (which is a safe default)

  `net.PK.BT_L2CAP`
  : Bluetooth protocol

  `net.PK.BT_RFCOMM`
  : Bluetooth protocol

`net.SocketIO(fk=net.FK.INET4, sk=net.SK.TCP, pk=net.PK.AUTO)`
: This type represents a network socket, which is an endpoint for sending/receiving data across a network. There are different types of sockets, but the most commons are the default arguments. You can manually specify the family, socket, and protocol used by

supplying them, they should be members of the enums `net.FK`, `net.SK`, and `net.PK` respectively. The dfeault is IPv4, TCP/IP, and automatic protocol.

This is a subtype of `io.BaseIO`, and implements the pattern fully. Additionally, this type has the following methods:

`net.SocketIO.bind(self, addr)`
Binds the socket to the given address

The type expected for `addr` depends on the {@ref net.FK, family kind} of socket that `self` is

`net.SocketIO.connect(self, addr)`
Connects the socket to the given address

The type expected for `addr` depends on the {@ref net.FK, family kind} of socket that `self` is

`net.SocketIO.listen(self, num=16)`
Begins listening for connections, and only allows `num` to be active at once before refusing connections

`net.SocketIO.accept(self)`
Accepts a new connection, returning a tuple of `(sock, name)`, where `sock` is a `net.SocketIO` object that can be read from and written to, and `name` is a string representing the client's name

4.6.1 net.http: Hyper Text Transfer Protocol

The HTTP networking submodule (`net.http`) is a submodule of the `net` module. Specifically, it provides HTTP utilities built on top of the rest of the networking stack.

`net.http.Request(method, uri, httpv, headers, body)`
This type represents an HTTP request

It has the following attibutes:

`net.http.Request.method`
A string representing the HTTP method

See here for a list of valid methods

`net.http.Request.uri`
A string representing the requested path. Includes a leading `/`.

For example, a `GET` request to `mysite.com/path/to/dir.txt` would have `'/path/to/dir.txt'` as the `.uri` component

You can use the functions `net.http.uriencode` and `net.http.uridecode` to encode/decode components of a URI (for example, replaces reserved characters with `%` escapes)

`net.http.Request.httpv`
A string representing the HTTP protocol version. This is almost always `'HTTP/1.1'`

`net.http.Request.headers`
A `dict` representing key-val entries for the headers

net.http.Request.body
A `bytes` representing the request body (may be empty)

net.http.Response(httpv, code, headers, body)
This type represents an HTTP response

It has the following attibutes:

net.http.Response.httpv
A string representing the HTTP protocol version. This is almost always `'HTTP/1.1'`

net.http.Response.code
An integer representing the status code. Common values are `200` (OK), `404` (NOT-FOUND)

net.http.Response.headers
A `dict` representing the key-val pairs of headers

net.http.Response.body
A `bytes` object representing the body of the response

net.http.Server(addr)
This type represents an HTTP server, which binds itself to `addr`.

Commonly, to host a local server, you will want to give `addr` as `("localhost", 8080)` (or whatever port you want to host on).

Once you've created a server, you should call `serverobj.serve()` (`$net.http.Server.serve()`) to serve forever in the current thread. When a new connection is requested, it spawns a new thread and serves each request in a new thread.

Example:

```
>>> s = net.http.Server(("localhost", 8080))
>>> s.serve() # Hangs in the current thread, but spawns new ones to handle requests
```

net.http.Server.serve(self)
Serve forever on the current thread, spawning new threads that call `net.http.Server.handle()` for each request

net.http.Server.handle(self, addr, sock, req)
The request callback, which is called everytime a request is made to the server

- `addr`: A string representing the client's address
- `sock`: The socket/socket-like IO (commonly an `net.SocketIO`) that can be used to communicate with the client
- `req`: A request object (specifically, of the type `net.http.Request`), holding the information about the request

This method should typically not write to `sock`. Instead, it should return either a string, bytes, or a `net.http.Response` object, which will then be automatically written to the socket afterwards.

Here's an example that just returns the path requested:

```
func handle(self, addr, sock, req) {
    ret "You asked for: %s" % (req.uri,)
}
```

net.http.uriencode(text)

This function takes a string, `text`, and encodes reserved characters with appropriate escape codes (see here)

ASCII characters which are not escaped are added to the output unchanged; all non-ASCII characters are converted to their UTF-8 byte sequence, and `%` escaped each byte sequence

For the inverse of this function, see `net.http.uridecode`

Examples:

```
>>> net.http.uriencode('hey there everyone')
'hey%20there%20everyone'
>>> net.http.uriencode('I love to eat \N[GREEK SMALL LETTER PI]')
'I%20love%20to%20eat%20%CF%80'
```

net.http.uridecode(text)

This function takes a string, `text`, and decodes reserved characters from appropriate escape codes (see here)

Escape sequences outside of the normal ASCII range are taken to be UTF8, and decoded as such

For the inverse of this function, see `net.http.uriencode`

Examples:

```
>>> net.http.uridecode('hey%20there%20everyone')
'hey there everyone'
>>> net.http.uridecode('I%20love%20to%20eat%20%CF%80')
'I love to eat  '
```

4.7 ffi: Foreign Function Interface

The foreign function interface module, `ffi`, provides functionality required to load and execute dynamically loaded code from other languages.

This module has definitions for C-style types (including pointers, structs and functions), and utilities to load libraries.

ffi.s8

Signed 8 bit integer type

ffi.u8

Unsigned 8 bit integer type

ffi.s16

Signed 16 bit integer type

ffi.u16

Unsigned 16 bit integer type

ffi.s32

Signed 32 bit integer type

ffi.u32

Unsigned 32 bit integer type

ffi.s64
: Signed 64 bit integer type

ffi.u64
: Unsigned 64 bit integer type

ffi.char
: Alias for `char` in C

ffi.uchar
: Alias for `unsigned char` in C

ffi.short
: Alias for `short` in C

ffi.ushort
: Alias for `unsigned short` in C

ffi.int
: Alias for `int` in C

ffi.uint
: Alias for `unsigned int` in C

ffi.long
: Alias for `int` in C

ffi.ulong
: Alias for `unsigned int` in C

ffi.longlong
: Alias for `long long` in C

ffi.ulonglong
: Alias for `unsigned long long` in C

ffi.size_t
: Alias for `size_t` in C

ffi.ssize_t
: Alias for `ssize_t` in C

ffi.float
: Floating point type: `float` in C

ffi.double
: Floating point type: `double` in C

ffi.longdouble
: Floating point type: `long double` in C

ffi.ptr[T=none]
: A pointer to another type. This is an example of a {@ref Templates, templated type}. For example, in C you may have the type `typedef int* int_p;` (for a pointer to an `int`). In this library, you can represent such a type with the expression `ffi.ptr[ffi.int]`

  By default, `ffi.ptr` acts like `ffi.ptr[none]`, which acts like a `void*` in C (i.e. pointer to anything)

  Pointer arithmetic is defined and adds the type of the size. For example:

```
>>> ffi.ptr[ffi.int](0x64)
0x64
>>> ffi.ptr[ffi.int](0x64) + 1
ffi.ptr[ffi.s32](0x68)
```

Also, dereferencing and assignment are supported through `[]`:

```
>>> val = ffi.int(4)
4
>>> valp = ffi.addr(val)
ffi.ptr[ffi.s32](0x557EE456A4F0)  # Address of `val`
>>> valp[0] = 5
5
>>> val  # Now, that original value changed
5
```

`ffi.func[ResultT=none, ArgsT=()]`

A function type, with a result type and a list of argument types. This is an example of a {@ref Templates, templated type}. For example, in C you may have the function type `int (*myfunc)(char x, short y)`. In this library, you can represent such a type with the expression `ffi.func[ffi.int, (ffi.char, ffi.short)]`

Variadic functions are supported via the `...` constant at the end of the arguments array. For example, the `printf` function in C has the type `ffi.func[ffi.int, (ffi.ptr[ffi.char], ...)`.

FFI functions can be called in kscript, just like a normal function. However, there are a few conversions that go on under-the-hood:

- Objects are casted to the function's signatures type (i.e. the template arguments)
- For variadic functions, extra arguments are expected to be either FFI types already, or a standard type (`int`, `float`, `str`, `bytes`) that can be converted automatically
- `str` and `bytes` objects are converted into a *mutable* `ffi.ptr[ffi.char]`. It is important that you don't pass them to a function which may mutate them! Use `ffi.fromUTF8` and `ffi.toUTF8` to convert them to mutable buffers

`ffi.struct[*members]`

A structure type, representing a `struct` type in C. This is an example of a {@ref Templates, templated type}.

Here are some examples:

```
""" C code
struct vec3f {
    float x, y, z;
};
"""

# FFI code
vec3f = ffi.struct[
    ('x', ffi.float),
    ('y', ffi.float),
    ('z', ffi.float),
]
```

```
# Sometimes, if you want to add customizability, you can create a new type:
type vec3f extends ffi.struct[
        ('x', ffi.float),
        ('y', ffi.float),
        ('z', ffi.float),
    ] {

    # You can even add methods!
    func mag(self) {
        ret m.sqrt(self.x * self.x + self.y * self.y + self.z * self.z)
    }
}

# You can create values like this
# vec3f(1, 2, 3)
# vec3f(1, 2, 3).y == 2.0
```

`ffi.DLL()`

Represents a dynamically loaded library, typically opened via the `ffi.open` function.

`ffi.DLL.load(name, of=ffi.ptr)`

Loads a symbol from the DLL (by looking it up) which is expected to be of type `of` (default: `void*` in C)

For example, the `puts` function can be loaded like so:

```
>>> libc.load('puts', ffi.func[ffi.int, (ffi.ptr[ffi.char],)])
```

`ffi.open(src)`

Opens `src`, which is expected to be a string representing the dynamic library. For example, `ffi.open('libc.so.6')` opens the C standard library on most systems.

Returns an `ffi.DLL`

`ffi.sizeof(obj)`

Computes the size (in bytes) of an object (or, if given a type, the size of objects of that type)

`ffi.addr(obj)`

Returns the address of the value in `obj`, assuming its type is an FFI-based type. The address returns the address of the start of the actual C-value in `obj`, which can be mutated and is only valid as long as `obj` is still alive

Return type is `ffi.ptr[type(obj)]`

`ffi.fromUTF8(addr)`

Returns a string created from UTF8 bytes at `addr` (assumed to be NUL-terminated)

`ffi.toUTF8(obj, addr=none, sz=-1)`

Converts `obj` (which is expected to be a string or bytes object) to UTF8 bytes, and stores in `addr` (default: allocate via `malloc()`)

If `sz > 0`, then that is the maximum size of the buffer (including NUL-terminator), and no more than `sz` bytes will be written

4.8 time: Times and Dates

This module, `time`, allows code to programmatically determine the current time, convert date-times to human readable formats, create timestamps, and reason about multiple dates.

`time.ISO8601`
: This is a string which is the format that can be passed to `time.format` and `time.parse` to format and parse ISO8601 format. This format is the preferred format for exchanges of dates and times.

`time.time()`
: Returns the number of seconds since the Epoch as a floating point number.

  The Epoch may depend on your platform, but is most commonly 1970-01-01. You can check when the epoch is by passing `0` to `time.format`:

```
>>> time.format(0.0)
'1970-01-01T00:00:00+0000'
```

`time.clock()`
: Returns the number of seconds since the process started as a floating point number.

`time.sleep(dur)`
: Causes the current thread to sleep for `dur` seconds, which should be a floating point number

  The exact accuracy of this function cannot be guaranteed, it depends on the platform function. For example, if `nanosleep()` or `usleep()` are available in the C library, this function will be accurate. At worst, this function will only sleep to the nearest second

`time.now()`
: Returns a `DateTime` referring to the current time, in UTC

  See `time.localnow` for local-equivalent function

`time.localnow()`
: Returns a `DateTime` referring to the current time, in the local timezone

  See `time.now` for UTC-equivalent function

`time.format(val=none, fmt=time.ISO8601)`
: Returns a string which is the time value `val` (default: `time.now()`, which may be a floating point number, or a `DateTime` object) formatted according to `fmt`.

  The format string is similar to `printf` syntax, but with different seperators:

  - `%%`: Literal `%`
  - `%Y`: The year, in full
  - `%y`: The year, modulo 100 (i.e. `1970` would result in `70`)
  - `%m`: The month of the year (starting at 1), zero-padded to 2 digits (i.e. February would be `02`)
  - `%B`: The month of the year, in the current locale, full
  - `%b`: The month of the year, in the current locale, abbreviated
  - `%U`: The week of the year as a decimal number (Sunday is `0`). Days before the first Sunday are week 0
  - `%W`: The week of the year as a decimal number (Monday is `0`). Days before the first Monday are week 0

- `%j`: The day of the year as a decimal number (starting with `001`), zero-padded to 3 digits
- `%d`: The day of the month as a decimal number (starting with `01`), zero-padded to 2 digits
- `%A`: The day of the week, in the current locale, full
- `%a`: The day of the week, in the current locale, abbreviated
- `%w`: The day of the week as an integer (Monday is `0`)
- `%H`: Hour (in 24-hour clock), zero-padded to 2 digits (`00`, …, `23`)
- `%I`: Hour (in 12-hour clock), zero-padded to 2 digits (`00`, …, `12`)
- `%M`: Minute, zero-padded to 2 digits (`00`, …, `59`)
- `%S`: Second, zero-padded to 2 digits (`00`, …, `59`)
- `%f`: Microsecond, zero-padded to 6 digits (`000000`, …, `999999`)
- `%z`: Timezone UTC offset in `(+|-)HHMM[SS.[ffffff]]`
- `%Z`: Timezone name (or empty if there was none)
- `%p`: Current locale's equivalent of `AM` and `PM`
- `%c`: Current locale's default date/time representation
- `%x`: Current locale's default date representation
- `%X`: Current locale's default time representation

`time.DateTime(obj=none, tz=none)`

This type represents a broken-down time structure comprised of the attributes humans commonly associate with a time. For example, the year, month, day, and so forth.

DateTimes can be created with the empty constructor, `time.DateTime()`, which is equivalent to the `time.now` function. Or, you can pass the first argument as a number of seconds since the Epoch (i.e. what is returned by `time.time`). For example, `time.DateTime(0.0)` will give you a DateTime representing the system Epoch.

The constructor also accepts another argument, `tz`, which is the timezone. If not given, or none, the resulting datetime is in UTC (which is to say, a reasonable default). To get a datetime in local time, you can pass `'local'` as the second argument. You can also give it a specific name of a timezone, and it will attempt to use that timezone.

This type is not a completely consistent datatype (as it must deal with things like daylight savings, leap seconds, leap year, and so forth), so it is recommended to use a float to capture absolute times, and deal with timestamps.

Here are some examples of creating datetimes in various ways (the output may differ based on your location, obviously!):

```
>>> time.DateTime()      # Current datetime, in UTC
<time.DateTime '2021-01-13T01:10:26+0000'>
>>> time.DateTime(0)     # Epoch, in UTC
<time.DateTime '1970-01-01T00:00:00+0000'>
>>> time.DateTime(0, "local") # Epoch, in current timezone
<time.DateTime '1969-12-31T19:00:00-0500'>
```

`time.DateTime.tse`

The time since Epoch, in number of seconds as a floating point number

`time.DateTime.tz`

The timezone, which may be a string (the name), or `none` if there was no timezone

`time.DateTime.year`

The year as an integer

time.DateTime.month
The month as an integer

time.DateTime.day
The day as an integer

time.DateTime.hour
The hour as an integer

time.DateTime.min
The minute as an integer

time.DateTime.sec
The second as an integer

time.DateTime.nano
The nanosecond as an integer

4.9 util: Common Utilities

This module, `util`, implements commonly used datastructures and algorithms that aren't in the builtins. While they are commonly used, they are not used frequently enough to use the global namespace and thus restrict developers from using those names in their own code. So, you can think of the `util` module as "builtins 2: electric boogaloo".

util.Queue(objs=none)
This type represents a queue, which can handle arbitrary objects. It can be created via the constructor, which accepts `objs` (default: none), which is expected to be an iterable containing the elements to initialize the queue from.

The main purpose of a queue over the builtin `list` type is that certain operations are much more efficient for a queue. For example, popping from the left and right is `O(1)`, whereas with a list they are `O(N)` and `O(1)` respectively (where `N` is the length of the collection). This has large consequences if you are doing the operation over and over for example – using a queue will reduce runtime drastically.

A queue is iterable just like a `list`, and it iterates in the same order as a list.

util.Queue.pop(self)
Pops from the front of the queue (i.e. the `0`th element)

Note: This can be confusing, since `list.pop` pops from the back of the list (i.e. the last element). Keep in mind the differences

util.Queue.push(self, *args)
Pushes all of `args` to the back of the queue

util.BST(objs=none)
This type represents a key-value mapping following the `dict` pattern, implemented using a Binary Search Tree (BST). Unlike `dict`, however, keys must be comparable (not neccessily hashable), and are stored in sorted order (as opposed to insertion order). This is important for algorithms which maintain a sorted list or mapping.

You can construct a BST with the constructor, which accepts a `dict`-like mapping, which inserts every key/value pair into a sorted mapping.

Values for keys can be retreived via indexing: `x[k]` gets the value associated with key `k` in the BST `x`, or throws a `KeyError` if no such key exists. Likewise, `x[k] = v` adds (or replaces) the value associated with key `k` in the BST `x` with value `v`.

You can iterate over the keys in a binary search tree as well:

```
>>> x = util.BST({2: 'a', 1: 'b'})
util.BST({1: 'b', 2: 'a'})
>>> for k in x, print (k, x[k])
1 b
2 a
```

Examples:

```
>>> x = util.BST({2: 'a', 1: 'b'})
util.BST({1: 'b', 2: 'a'})
>>> 1 in x
true
>>> 3 in x
false
>>> x[0] = 'hey'
>>> x
util.BST({0: 'hey', 1: 'b', 2: 'a'})
```

`util.Bitset(objs=none)`

This type represents a bitset (also called "bit map", "bit array", etc). The general idea is that it can store whether positive integers are in a set or not based on a single integer mask. This type is supposed to behave exactly like the builtin `set` type, except it only supports positive integers, and uses `O(max(X))` memory (where `X` are the elements in the set). Sets use `O(N)` memory, where `N` is the number of elements in the set.

Bitsets can be created with an iterable `objs` (default: none), in which case all elements are converted to an integer and added to the set. Additionally, a bitset can be created with a single integer, which represents the bit mask of the entire set (see below).

If we look at a number decomposed into bits, we can say that if the `i`th bit is set to `1`, then `i` is in the set, and otherwise, `i` is not in the set. For example, the integer `0b11011` corresponds to the integers `0`, `1`, `3`, and `4`.

You can convert a bitset into the corresponding integer like so:

```
>>> util.Bitset([0, 1, 3, 4]) as int
27
>> bin(27)    # Check binary notation
0b11011
```

The main advantage of using a bitset over a normal set with integers is speed of common operations. For example, intersection, union, difference, and so forth can be computed extremely efficiently, perhaps 10x faster. So, this type is available for those use cases.

`util.Graph(nodes=none, edges=none)`

This type is currently being implemented, so this documentation is incomplete.

4.10 gram: Grammar Regonition

Grammar Module

The grammar module (`gram`) provides commonly needed types and algorithms for dealing with computer grammars (for example, descriptions of programming language syntax)

`gram.Lexer(patterns, src=os.stdin)`

This type represents a lexer/tokenizer which token rules are defined as either `str` literals or `regex` patterns.

The constructor takes an iterable of rules – each rule is expected to be a tuple of length 2, containing `(pattern, action)`. `pattern` may be a `regex` or `str`; if it is a string, then the token matches that string literal exactly, and otherwise the token matches the regex pattern. `action` may be a function (in which case the result of a token being found is the result of `action(tokenstring)`). Otherwise, it is may be an integral object, which is the token kind. Commonly, these may be members of an enumeration meant to represent every token kind in a given context. Otherwise, `action` is expected to be `none`, in which case the potential token is discarded and the characters that made up that token are ignored. The rule that is chosen is that which generates the longest match from the current position in a file. If two rules match the same length, then the one given first in the `rules` variable is used first.

The second argument, `src`, is expected to be an object similar to `io.BaseIO` (default is `$os.stdin`). This is the source from which characters are taken.

This is hard to grasp abstractly – here is an example recognizing numbers and words, and ignoring everything else:

```
>>> L = gram.Lexer([
...     (`[:alpha:]+`,   0),
...     (`[:digit:]+`,   1),
...     (`.`,     none).
...     (`\n`,    none)
... ], io.StringIO("hey 123 456 test"))
gram.Lexer([(regex('[[:alpha:]_][[:alpha:]_[:digit:]]*'), 0), (regex('\\d+'), 1), (
>>> next(L)
gram.Token(0, 'hey')
>>> next(L)
gram.Token(1, '123')
>>> next(L)
gram.Token(1, '456')
>>> next(L)
gram.Token(0, 'test')
>>> next(L)
OutOfIterException:
Call Stack:
  #0: In '<expr>' (line 1, col 1):
next(L)
  #1: In next(obj) [cfunc]
  #2: In gram.Lexer.__next(self) [cfunc]
In <thread 'main'>
```

You can also iterate over a lexer to produce the token stream:

```
>>> L = gram.Lexer([
...     (`[:alpha:]+`,   0),
...     (`[:digit:]+`,   1),
```

```
...     (`.`,     none).
...     (`\n`,    none)
... ], io.StringIO("hey 123 456 test"))
gram.Lexer([(regex('[[:alpha:]_][[:alpha:]_[:digit:]]*'), 0), (regex('\\d+'), 1), (
>>> for tok in L, print(repr(tok))
gram.Token(0, 'hey')
gram.Token(1, '123')
gram.Token(1, '456')
gram.Token(0, 'test')
```

5 Complete Syntax Reference

This section describes the syntax of the kscript language. It explains the actual formal specs of what is and what is not valid kscript code.

5.1 Formal Grammar EBNF

EBNF is a notation to formalize computer grammars. In this page, the grammar of the kscript language is described using an EBNF-like syntax:

```
(* Entire program/file *)
PROG    : STMT*

(* Newline/break rule *)
N       : '\n'
        | ';'

(* Block (enclosed in '{}') *)
B       : '{' STMT* '}'

(* Block (B) or comma statement *)
BORC    : B
        | ',' STMT

(* Block (B) or statement *)
BORS    : B
        | STMT

(* Statement, which does not yield a value *)
STMT    : 'import' NAME N
        | 'ret' EXPR? N
        | 'throw' EPXR? N
        | 'break' N
        | 'cont' N
        | 'if' EXPR BORC ('elif' BORC)* ('else' EXPR BORS)?
        | 'while' EXPR BORC ('else' EXPR BORS)?
        | 'for' EXPR BORC
        | 'try' BORS ('catch' EXPR (('as' | '->') EXPR)?)* ('finally' BORS)?
        | EXPR N
        | N
```

```
(* Expression, which does yield a value *)
EXPR        : E0

(* Precedence rules *)

E0      : E1 '=' E0
        | E1 '&=' E0
        | E1 '^=' E0
        | E1 '|=' E0
        | E1 '<<=' E0
        | E1 '>>=' E0
        | E1 '+=' E0
        | E1 '-=' E0
        | E1 '*=' E0
        | E1 '@=' E0
        | E1 '/=' E0
        | E1 '//=' E0
        | E1 '%=' E0
        | E1 '**=' E0
        | E1

E1      : E2 'if' E2 ('else' E1)?
        | E2

E2      : E2 '??' E3
        | E3

E3      : E3 '||' E4
        | E4

E4      : E4 '&&' E5
        | E5

E5      : E5 '===' E6
        | E5 '==' E6
        | E5 '!=' E6
        | E5 '<' E6
        | E5 '<=' E6
        | E5 '>' E6
        | E5 '>=' E6
        | E5 'in' E6
        | E5 '!in' E6
        | E6

E6      : E6 'as' E7
        | E7

E7      : E7 '|' E8
        | E8
```

```
E8      : E8 '^' E9
        | E9

E9      : E9 '&' E10
        | E10

E10     : E10 '<<' E11
        | E10 '>>' E11
        | E11

E11     : E11 '+' E12
        | E11 '-' E12
        | E12

E12     : E12 '*' E13
        | E12 '@' E13
        | E12 '/' E13
        | E12 '//' E13
        | E12 '%' E13
        | E13

E13     : E14 '**' E13
        | E14

E14     : '++' E14
        | '--' E14
        | '+' E14
        | '-' E14
        | '~' E14
        | '!' E14
        | '?' E14
        | E15

E15     : ATOM
        | '(' ')'
        | '[' ']'
        | '{' '}'
        | '(' ELEM (',' ELEM)* ','? ')'
        | '[' ELEM (',' ELEM)* ','? ']'
        | '{' ELEM (',' ELEM)* ','? '}'
        | '{' ELEMKV (',' ELEMKV)* ','? '}'
        | 'func' NAME? ('(' (PAR (',' PAR)* ','?)? ')')? B    (* Func constructor *)
        | 'type' NAME? ('extends' EXPR)? B                    (* Type constructor *)
        | 'enum' NAME? B                                      (* Enum constructor *)
        | E15 '.' NAME
        | E15 '++'
        | E15 '--'
        | E15 '(' (ARG (',' ARG)*)? ','? ')'
        | E15 '[' (ARG (',' ARG)*)? ','? ']'
```

```
(* Atomic element of grammar (expression which is single token) *)
ATOM    : NAME
        | STR
        | REGEX
        | INT
        | FLOAT
        | '...'

(* Valid argument to function call *)
ARG     : '*' EXPR
        | EXPR

(* Valid parameter to a function *)
PAR     : '*' NAME
        | NAME ('=' EXPR)?

(* Valid argument to container constructor (expression, or expand expression) *)
ELEM    : '*' EXPR
        | EXPR

(* Valid argument to key-val container constructor *)
ELEMKV  : EXPR ':' EXPR

(* Tuple literal *)
TUPLE   : '(' ','? ')'
        | '(' ELEM (',' ELEM)* ','? ')'

(* List literal *)
LIST    : '[' ','? ']'
        | '[' ELEM (',' ELEM)* ','? ']'

(* Set literal (no empty set, since that conflicts with dict) *)
SET     : '{' ELEM (',' ELEM)* ','? '}'

(* Dict literal *)
DICT    : '{' ','? '}'
        | '{' ELEMKV (',' ELEMKV)* ','? '}'

(* Function constructor *)
FUNC    : 'func' NAME? ('(' (PAR (',' PAR)*)? ','? ')')? B

(* Type constructor *)
TYPE    : 'type' NAME? ('extends' EXPR)? B

(* Enum constructor *)
ENUM    : 'enum' NAME? B

(* Token kinds described as literals *)
NAME    : ? unicode identifier ?
```

```
STR     : ? string literal ?
REGEX   : ? regex literal ?
INT     : ? integer literal ?
FLOAT   : ? floating point literal ?
```

5.2 Expressions

In kscript, many syntax elements are **expressions**, which means that they will result in a value after being evaluated. You can always assign the result of an expression to a variable, element index, attribute, or any other destination where you may store in any object. They can be used within other expressions (albeit, sometimes requiring `()` due to order-of-operations).

In contrast to most languages, Function Definitions and Type Definitions are expressions (in most languages, they are statements and do not yield a value). You can, of course, use them like a statement (i.e. not embedded in another expression), but you can also return them, or assign them locally.

5.2.1 Integer Literal

This is the syntax for constructing literal integers (of type `$int`). You can specify the base-10 digits within a kscript program, and it will be interpreted as an integer. You can also use a prefix for other notations – see the table below for a list of valid ones:

`0d` Decimal notation involves the prefix `0d` or no prefix followed by a sequence of base-10 digits (`0-9`)

`0b` Binary notation involves the prefix `0b` or `0B` followed by a sequence of base-2 digits (`0` or `1`)

`0o` Octal notation involves the prefix `0o` or `0O` followed by a sequence of base-8 digits (`0-7`)

`0x` Hexadecimal notation involves the prefix `0x` or `0X` followed by a sequence of base-16 digits (`0-9`, `a-f/A-F`)

Regardless of the notation, the result is an `$int` object with the specified value. Also, note that there are no "negative integer literals" – only a positive one with a `-` operator, which causes negation.

Examples:

```
>>> 123       # Base-10
123
>>> 255       # Base-10
255

>>> 0x7B      # Base-16
123
>>> 0xFF      # Base-16
255

>>> 0o173     # Base-8
123
>>> 0o377     # Base-8
255
```

```
>>> 0b1111011 # Base-2
123
>>> 0b11111111 # Base-2
255
```

5.2.2 Float Literal

This is the syntax for constructing literal floating point numbers (of type `$float`). You can specify the base-10 digits within a kscript program, including a `.` for the whole number/fractional separator (which differentiates `$float` literals from Integer Literal), and it will be interpreted as a real number, represented as accurately as the machine precision can (see `$float.EPS`). Additionally, an exponent is allowed (see: scientific notation) with the `e` or `E` characters. You can also use a prefix for other notations – see the table below for a list of valid ones:

`0d` Decimal notation involves the prefix `0d` or no prefix followed by base-10 digits (`0-9`)

`0b` Binary notation involves the prefix `0b` or `0B` followed by base-2 digits (`0` or `1`). Must include a `.` as a separator. Use `p` or `P` for base-2 power.

`0o` Octal notation involves the prefix `0o` or `0O` followed by base-8 digits (`0-7`). Must include a `.` as a separator. Use `p` or `P` for base-2 power.

`0x` Hexadecimal notation involves the prefix `0x` or `0X` followed by base-16 digits (`0-9`, `a-f`/`A-F`). Must include a `.` as a separator. Use `p` or `P` for base-2 power.

In addition to the digits, there are also two builtin names `inf` and `nan`, which represent positive infinity and not-a-number respectively.

Also, note that there are no "negative float literals" – only a positive one with a `-` operator, which causes negation.

Regardless of the notation, the result is a `$float` object with the specified value.

Examples:

```
>>> 123.0     # Base-10
123.0
>>> 255.0     # Base-10
255.0
>>> 100.75    # Base-10
100.75

>>> 0x7B.0    # Base-16
123.0
>>> 0xFF.0    # Base-16
255.0
>>> 0x64.C    # Base-16
100.75

>>> 0o173.0   # Base-8
123.0
>>> 0o377.0   # Base-8
255.0
>>> 0o144.6   # Base-8
100.75
```

```
>>> 0b1111011.0 # Base-2
123.0
>>> 0b11111111.0 # Base-2
255.0
>>> 0b1100100.11 # Base-2
100.75
```

Scientific notation examples:

```
>>> 1.234e3
1234
>>> 1234e-3
1.234
>>> 1e9
1000000000.0
```

5.2.3 Complex Literal

This is the syntax for constructing literal `$complex` values. Specifically, you can only construct imaginary literals – you need to use the `+` operator to create a complex number with real and imaginary components.

Complex literals are created by placing an `i` or `I` directly after an Integer Literal or Float Literal, which results in a complex number with `0.0` as the real component, and the integer or floating point value as the imaginary component. Note that `$complex` objects have both components as `$float` values.

Examples:

```
>>> 1i          # Base-10
1.0i
>>> 123i        # Base-10
123.0i
>>> 12+34i      # Base-10
(12.0+34.0i)

>>> 0x1i        # Base-16
1.0i
>>> 0x7Bi       # Base-16
123.0i
>>> 0xC+0x22i   # Base-16
(12.0+34.0i)
```

5.2.4 String Literal

This is the syntax for constructing literal `$str` values. The basic syntax is a beginning quote character (one of `'`, `"`, `'''`, `"""`), followed by the contents of the string, and then an ending quote character that matched the one the string started with. The contents of the string can be either character literals, or escape codes.

Escape sequences:

`\\` A literal \

`\'` A literal '

`\"` A literal "

`\a`

ASCII `BEL` (bell/alarm)

`\b` ASCII `BS` (backspace)

`\f` ASCII `FF` (formfeed)

`\n` ASCII `LF` (newline/linefeed)

`\r` ASCII `CR` (carriage return)

`\t` ASCII `HT` (horizontal tab)

`\v` ASCII `VT` (vertical tab)

`\xXX`
Single byte, where `XX` are the 2 hexadecimal digits of the codepoint

`\uXXXX`
Unicode character, where `XXXX` are the 4 hexadecimal digits of the codepoint

`\UXXXXXXXX`
Unicode character, where `XXXXXXXX` are the 8 hexadecimal digits of the codepoint

`\N[XX...X]`
Unicode character, where `XX...X` is the name of the Unicode character

Examples:

```
>>> '\x61'
'a'
>>> '\u0061'
'a'
>>> '\U00000061'
'a'
>>> '\N[LATIN SMALL LETTER A]'
'a'
```

5.2.5 List Literal

This is the syntax for constructing literal `$list` objects. A `$list` is a mutable collection.

List literals are created by surrounding the elements of the list with `[` and `]`, with `,` in between each element. Optionally, an additional `,` may be added after all of them. Items within a list literal may span across lines.

An empty list can be created with either `[]` or `[,]`.

List literals also support unpacking (`*` before an element) and comprehension (`for` and optionally `if`).

Examples:

```
>>> []
[]
```

```
>>> [,]
[]
>>> [1, 2, 3]
[1, 2, 3]
>>> ["Any", "Type", "CanBeStored"]
['Any', 'Type', 'CanBeStored']
>>> [*"abcd"]
['a', 'b', 'c', 'd']
```

5.2.6 Tuple Literal

This is the syntax for constructing literal `$tuple` objects. A `$tuple` is an immutable collection.

Tuple literals are created by surrounding the elements of the tuple with `(` and `)`, with `,` in between each element. Optionally, an additional `,` may be added after all of them. Tuples with one element must end with a comma (i.e. `(a,)`).

The empty tuple can be created via either `()` or `(,)`.

Tuple literals also support unpacking (`*` before an element) and comprehension (`for` and optionally `if`).

Examples:

```
>>> ()
()
>>> (,)
()
>>> (1, 2, 3)
(1, 2, 3)
>>> ("Any", "Type", "CanBeStored")
('Any', 'Type', 'CanBeStored')
>>> (*"abcd")
('a', 'b', 'c', 'd')
```

5.2.7 Set Literal

This is the syntax for constructing literal `$set` objects. A `$set` is a mutable collection.

Set literals are created by surrounding the elements of the set with `{` and `}`, with `,` in between each element. Optionally, an additional `,` may be added after all of them.

Due to a conflict with Dict Literal, the empty set must be created via calling the `$set` type: `set()`.

Set literals also support unpacking (`*` before an element) and comprehension (`for` and optionally `if`).

Examples:

```
>>> set()
set()
>>> {1, 2, 3}
{1, 2, 3}
>>> {"Any", "Type", "CanBeStored"}
{'Any', 'Type', 'CanBeStored'}
```

```
>>> {*"abcd"}
{'a', 'b', 'c', 'd'}
>>> {3, 2, 1, 2, 3}    # Duplicate elements are not added
{3, 2, 1}
```

5.2.8 Dict Literal

This is the syntax for constructing literal `$dict` objects. A `$dict` is a mutable collection.

Dict literals are created by surrounding the key-value pairs of the dict with `{` and `}`, with `:` separating the key and value, and with `,` in between each element. Optionally, an additional `,` may be added after all of them.

The empty dictionary can be created via either `{}` or `{,}`.

Dict literals also support comprehension (`for` and optionally `if`).

Examples:

```
>>> {}
{}
>>> {'a': 1, 'b': 2, 'c': 3}
{'a': 1, 'b': 2, 'c': 3}
>>> {'a': 1, 'b': 2, 'c': 3, 'a': 4} # Duplicate elements update the value, but do not a
{'a': 4, 'b': 2, 'c': 3}
```

5.2.9 Lambda Function Expression

This is the syntax for constructing `$func` objects given a lambda expression.

Lambda expressions are created from a variable, or Tuple Literal, followed by a right arrow (`->`), and then followed by an expression. The basic form is `a -> b`, where `a` are the parameters, and `b` is the expression to evaluate when the function is called.

Parameters may be assigned a default, but they must be within a tuple literal.

Examples:

```
>>> x -> x + 2
<func '<lambda>(x)'>
>>> (x -> x ** 2)(3)
9
>>> foo = (x, y=2) -> x + y
<func '<lambda>(x, y=2)'>
>>> foo(10)
12
>>> foo(10, 4)
14
```

5.2.10 Function Definition

This is the syntax for constructing `$func` objects with the `func` keyword. In kscript, function definition is an **expression**, not only a statement.

The basic syntax for defining a function begins with the `func` keyword, optionally followed by a valid identifier as the name of the function, optionally followed by the parameters (`(`, then

all the arguments, then `)`). Finally, it expects the body of the function within `{` and `}`.

Examples:

```
# Anonymous function, is not assigned to any name
func {
    a()
    b()
    ret val
}

# Named function (assigned to the local name `foo`)
func foo {
    a()
    b()
    ret val
}

# Named function, takes 2 arguments
func foo(x, y) {
    ret x + y
}

# Named function, takes 1 or 2 arguments (if only `x` is given, `y` defaults to `1`)
func foo(x, y=1) {
    ret x + y
}

# Named function, takes 1 or more arguments (`y` is a list of all other arguments given
func foo(x, *y) {
    ret x + len(y)
}

# Named function, takes 2 or more arguments (`y` is a list of all other arguments given
func foo(x, *y, z) {
    ret x + len(y) + z
}
```

You can also create a Lambda Expression, which is a similar concept but different syntax.

5.2.11 Type Definition

This is the syntax for constructing `$type` objects with the `type` keyword. In kscript, type definition is an **expression**, not only a statement.

The basic syntax for defining a type begins with the `type` keyword, optionally followed by a valid identifier as the name of the type, optionally followed by the keyword `extends` and an expression giving the base type (default: `$object`). It should end with a block of statements beginning with `{` and ending with `}`.

Examples:

```
# Anonymous type
```

```
type {
    ...
}

# Named type
type MyType {
    ...
}

# Named type which is a subtype of `list`
type MyType extends list {
    ...
}
```

5.2.12 Operators

Operators are used to represent operations on other expressions (called "operands"). There are a few different kinds of operators:

- Unary operators (such as `+x`, `-x`, etc.), which take 1 operand
- Binary operators (such as `x+y`, `x-y`, etc.), which take 2 operands
- Ternary operators (such as `x if y else z`), which take 3 operands

Operators in kscript are divided into precedence levels. For example, `a + b + c` is parsed as `(a + b) + c`, but `a + b * c` is parsed as `a + (b * c)`.

Operator precedence (lowest first):

1. `=, &=, ^=, |=, <<=, >>=, +=, -=, *=, @@=, /=, //=, %=, **=`
2. `if/else`
3. `??`
4. `||`
5. `&&`
6. `===, ==, !=, >, <=, >, >=, in, !in`
7. `|`
8. `^`
9. `&`
10. `<<, >>`
11. `+, -`
12. `*, @@, /, //, %`
13. `**`
14. `~, !, ++ (pre), -- (pre), + (unary), - (unary)`
15. `., ++ (post), -- (post)`

See the EBNF section for details on each operator.

5.3 Statements

These elements do not yield a value, and are typically used for control flow.

In contrast to most languages, Function Definitions and Type Definitions are expressions (in most languages, they are statements and do not yield a value). You can, of course, use them like a statement (i.e. not embedded in another expression), but you can also return them, or assign them locally.

5.3.1 Expression Statement

The expression statement is an **expression**, which has a line break after it, or, equivalently, a semicolon (`;`). You can write code without using semicolons (`;`), and it is recommended to not use them.

Examples:

```
# This is the best, it is clear and readable
foo()
bar()

# Don't do this, the ';' are unnecessary
foo();
bar();

# Don't do this, it's less readable
foo(); bar()
```

5.3.2 Assert Statement

The `assert` statement is an example of assertion syntax for the kscript language. It instructs the program to evaluate a conditional expression, check whether it is truthy (see `$bool`), and if it is not truthy, throws an `$AssertError` up the call stack.

Example:

```
# Evaluates `x` and asserts that it is true
assert x
```

5.3.3 Cont Statement

The `cont` statement prematurely terminates the innermost loop (which may be a while statement, or for statement) and retries the next iteration.

Example:

```
# infinite loop
while true {
    # Stops executing now, and continues the loop
    cont
    # This code never runs
    x = 3
}
```

See also the Break Statement.

5.3.4 Break Statement

The `break` statement prematurely terminates the innermost loop (which may be a while statement, or for statement) and then stops executing the loop.

Example:

```
# looks like an infinite loop, but is not since the `break` statement will terminate it
while true {
```

```
    # Stops executing now, and exits the loop
    break
    # This code never runs
    x = 3
}
```

See also the Cont Statement.

5.3.5 Ret Statement

The `ret` statement is an example of return syntax for the kscript language. It instructs the program to return a value (or `none`) from the current function, and stop executing in that function.

Example:

```
# Returns the value `a` and stops executing the current function
ret a

# Equivalent to `ret none`
ret
```

5.3.6 Throw Statement

The `throw` statement is an example of exception handling syntax for the kscript language. It instructs the program to throw a value up the call stack, which searches through the current functions executing in the thread, until it finds a try statement.

Example:

```
# Throws the value `a` up the call stack
throw a
```

Usage:

```
func foo(x) {
    if x == 0 {
        throw Exception("'x' should never be 0")
    }
    ...
}

foo(1)
foo(2)
foo(0) # This crashes and prints: Exception: 'x' should never be 0

try {
    foo(0)
} catch as err {
    print ("I got:", type(err))
}
```

5.3.7 If Statement

The `if` statement is an example of a conditional statement, which evaluates a condition, and then based on the result, either runs another statement, or does not.

Syntax:

```
if a {
    b
}
```

You can use `elif` and `else` clauses:

```
if a {
    b
} elif c {
    d
} elif e {
    f
} else {
    g
}
```

Inline `if` syntax (single statement body):

```
if a, b
```

5.3.8 While Statement

The `while` statement is an example of a conditional loop, which evaluates a condition, and then based on the result, either runs another statement and then retries the condition, or exits the loop.

Syntax:

```
while a {
    b
}
```

You can use `elif` and `else` clauses:

```
while a {
    b
} elif c {
    d
} else {
    e
}
```

Inline `while` syntax (single statement body):

```
while a, b
```

5.3.9 For Statement

The `for` statement is an iterator-based for loop, which iterates over elements of an iterable.

Syntax:

```
for a in b {
    c
}
```

You can use `elif` and `else` clauses:

```
for a in b {
    c
} elif d {
    e
} else {
    f
}
```

Inline `for` syntax (single statement body):

```
for a in b, c
```

5.3.10 Try Statement

The `try` statement (also called `try/catch` statement) is the exception handling syntax for kscript. It allows exceptions thrown with the throw statement while executing the body of the `try` statement to be "caught", and then handled appropriately.

Syntax:

```
try {
    a
} catch NameError {
    # Only selected if the exception thrown was a subtype of `NameError`
} catch NameError as err {
    print (err)
} catch (NameError, SizeError) as err {
    # Matches any exception which is a subtype of any of the elements in the tuple
} catch {
    # Selected for any thrown exception
} catch as err {
    # Selected for any thrown exception, and captures the thrown object as `err`
}
```

Optionally, a `finally` clause:

```
try {
    a
} catch {
    ...
} finally {
    b
}
```

5.3.11 Import Statement

To import a module (for example, one of the builtin modules), you can use the `import` statement. If there was an error finding or importing the module, an `$ImportError` is thrown.

Syntax:

```
import os
import net
```

6 Implementation Details

TODO: add them here